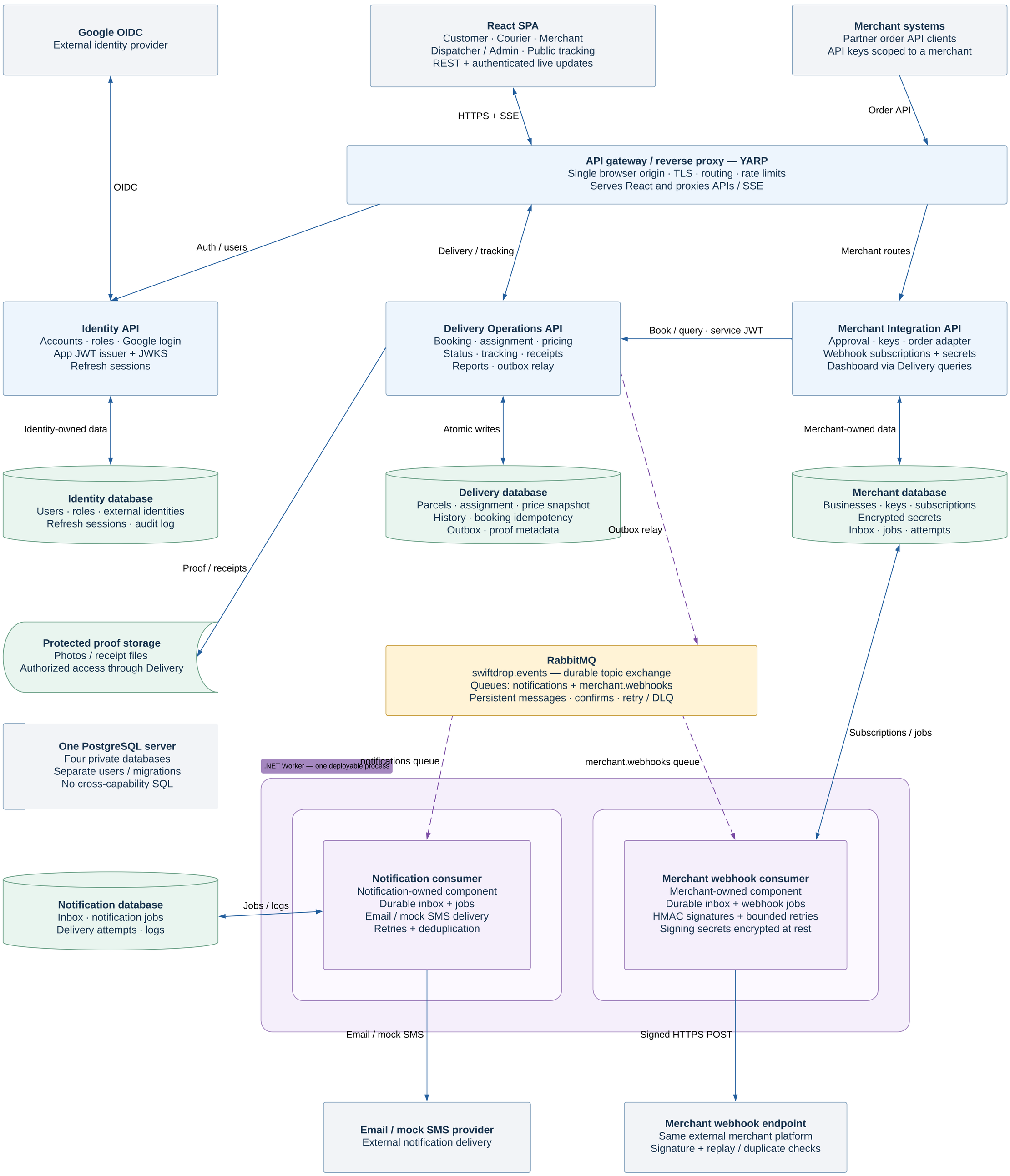
SwiftDrop — Final Application Architecture

Three capability APIs + one Worker host | C# / ASP.NET Core | React | RabbitMQ | PostgreSQL



Boundaries, reliability and deployment

Each API separates endpoints, application use cases, domain rules and infrastructure adapters. JWT validation and resource authorization run locally.
Delivery atomically commits business state and outbox; its relay publishes with confirms. Consumers commit inbox + jobs before acknowledging.
Merchant webhook data stays Merchant-owned even though the processor runs beside notifications. Each component uses its own scoped database connection.
Blue solid arrows: HTTP / data access. Purple dashed arrows: asynchronous delivery. Replies are implied except the explicit live-update path.
Docker Compose baseline: React, gateway, three APIs, Worker, RabbitMQ and PostgreSQL. Database, broker and proof storage are persistent.
Live updates: fetch-based SSE + JWT, token refresh and reconnect/refetch. Start with one Delivery replica; add explicit fan-out when scaling.
Redis and Kubernetes are optional extensions. Gateway/broker/database and the shared Worker remain operational failure boundaries.